Lecture4 数据智能计算架构设计（2）



于晗

han_yu@sjtu.edu.cn

目录

- 1 数据智能计算架构回顾
- 2 分布式机器学习框架
- 3 小结

目录

- 1 数据智能计算架构回顾
- 2 分布式机器学习框架
- 3 小结

深度学习框架

















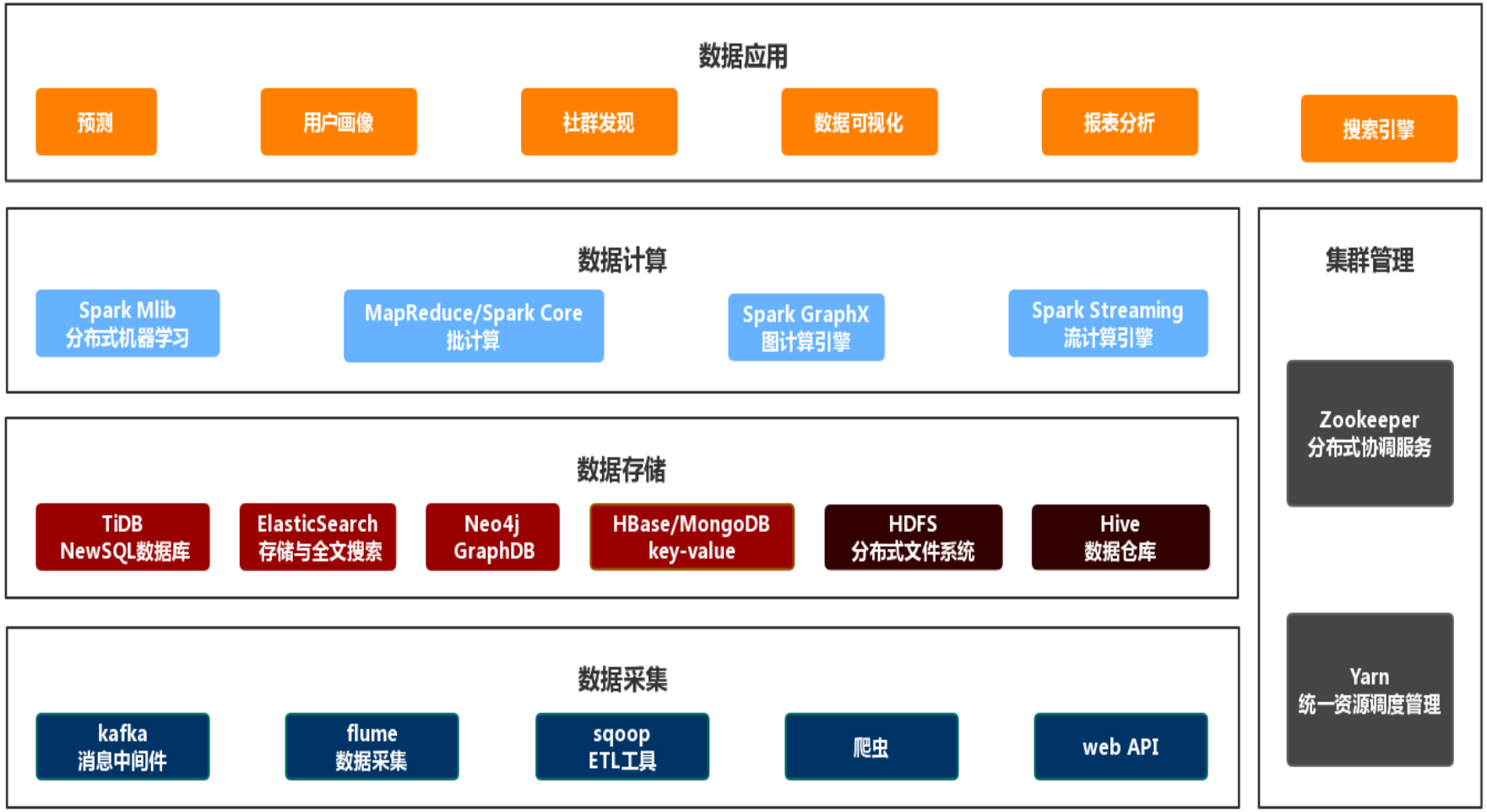
机器学习框架





1.1 架构

一个典型的大数据平台架构



1.2 大数据分析算法

- 根据大数据分析算法对实时性的要求可以也分为三类：
  - **实时分析算法**：这类分析算法适用实时获取的数据，响应时间约束为秒级甚至毫秒级。
  - **弱实时分析算法：**这类分析算法面向有用户参与分析决策的分析任务，不要求实时响应，但是也存在响应时间约束，响应时间约束从分钟到小时。
  - **非实时分析算法：**这类分析算法适用数据仓库中的大规模数据，响应时间约束相对宽松，可以达到天甚至月。
- 根据大数据分析算法的功用可以分为四类：
  - **分类算法**
  - **聚类算法**
  - **关联算法**
  - **回归算法**

目录

- 1 数据智能计算架构回顾
- 2 分布式机器学习框架
- 3 小结

深度学习框架

















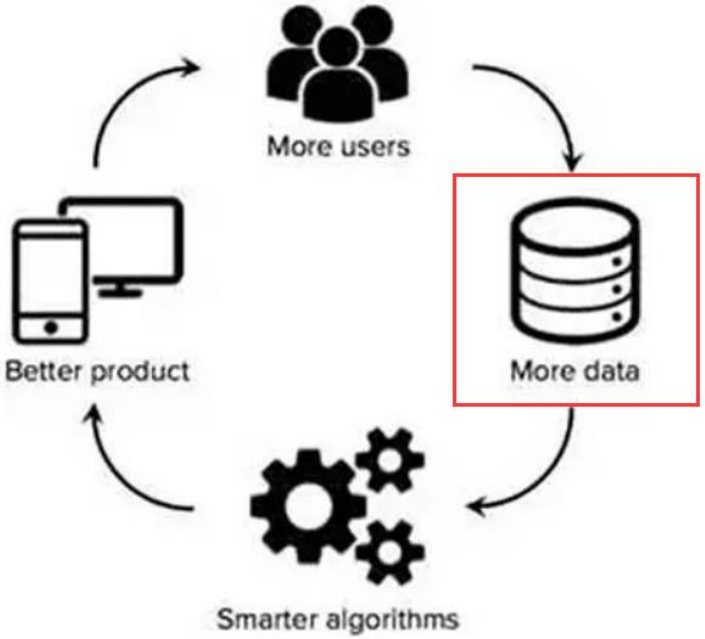
机器学习框架





2.1 为什么要使用机器学习框架?

- 机器学习（尤其是深度学习）最近已经在语音识别、图像识别、自然语言处理和推荐/搜索引擎等方面取得了变革性的成功。现有的机器学习模型往往涉及巨大的数据集，且需要快速的迭代和便捷的管理。
- 机器学习框架的优点
  - 更高的效率:
    - 机器学习框架可以利用现有的硬件资源进行分布式计算，从而大幅度提高计算速度和效率。
  - 更低的错误率:
    - 机器学习框架通常使用已经经过验证和优化的算法，可以避免手动实现算法时常见的错误。
  - 更容易的代码维护和管理:
    - 使用机器学习框架可以使代码更加模块化和可维护，可以方便地进行代码版本控制和团队协作。
  - 更快的模型迭代:
    - 机器学习框架提供了许多优化的算法和工具，可以更快地实现模型训练和迭代，从而更快地适应新的数据和需求。



2.2 机器学习/深度学习框架发展历程

- 单机时代
- 2007年: Sci-kit learn:
  - 公开发布：2010
  - Python 的机器学习（ML）库
  - 包含大量“经典”机器学习算法，例如线性回归和逻辑回归、支持向量机、随机森林、k-means、梯度提升
  - 到今天仍然非常成功
- 大数据机器学习时代
- 由于数据量大幅增加，研究人员开发了分布式算法：
  - 仍然属于“经典”机器学习，例如逻辑回归、协同过滤
  - 2009 年：Mahout：基于 Apache Hadoop 的机器学习库
    - 每次迭代都会在磁盘上读取/写入数据
  - 2011 年：MLlib：Apache Spark 的机器学习库
    - 比 Mahout 快得多：无需读写磁盘
    - 仍然是分布式“经典”机器学习算法的首选库

2.2 机器学习/深度学习框架发展历程

- 神经网络单机库
- 2007年：Theano
  - 由蒙特利尔学习算法研究所（MILA）开发
  - 初始不支持 GPU

theano

- 为什么支持 GPU 很重要?
  - 神经网络基本需要矩阵乘法
    - 空间复杂度：O(N^2) – 计算复杂度：O(N^3)
    - 因此，计算复杂度随输入超线性增长

2.2 机器学习/深度学习框架发展历程

- 神经网络单机库
- 2007年：Theano
  - 由蒙特利尔学习算法研究所（MILA）开发
  - 初始不支持 GPU
- 2014 年：Caffee
  - 由伯克利视觉和学习中心开发
  - 支持 GPU，一些流行的神经网络，例如 AlexNet
- 2016 年：PyTorch
  - 由 Facebook 开发
    - 基于 Torch（始于 2002 年，但现已不再活跃）
  - 最初为单机，最近已实现分布式

theano

Caffe

PyTorch

2.2 机器学习/深度学习框架发展历程

- 神经网络：分布式系统
- 2015年：Tensorflow
  - 由谷歌大脑开发
  - 今天的最受欢迎的机器学习库
- 2015：MXNet
  - 最初由 UW 和其他机构开发；现在由 AWS 负责





2.2 机器学习/深度学习框架发展历程

Compute system requirements

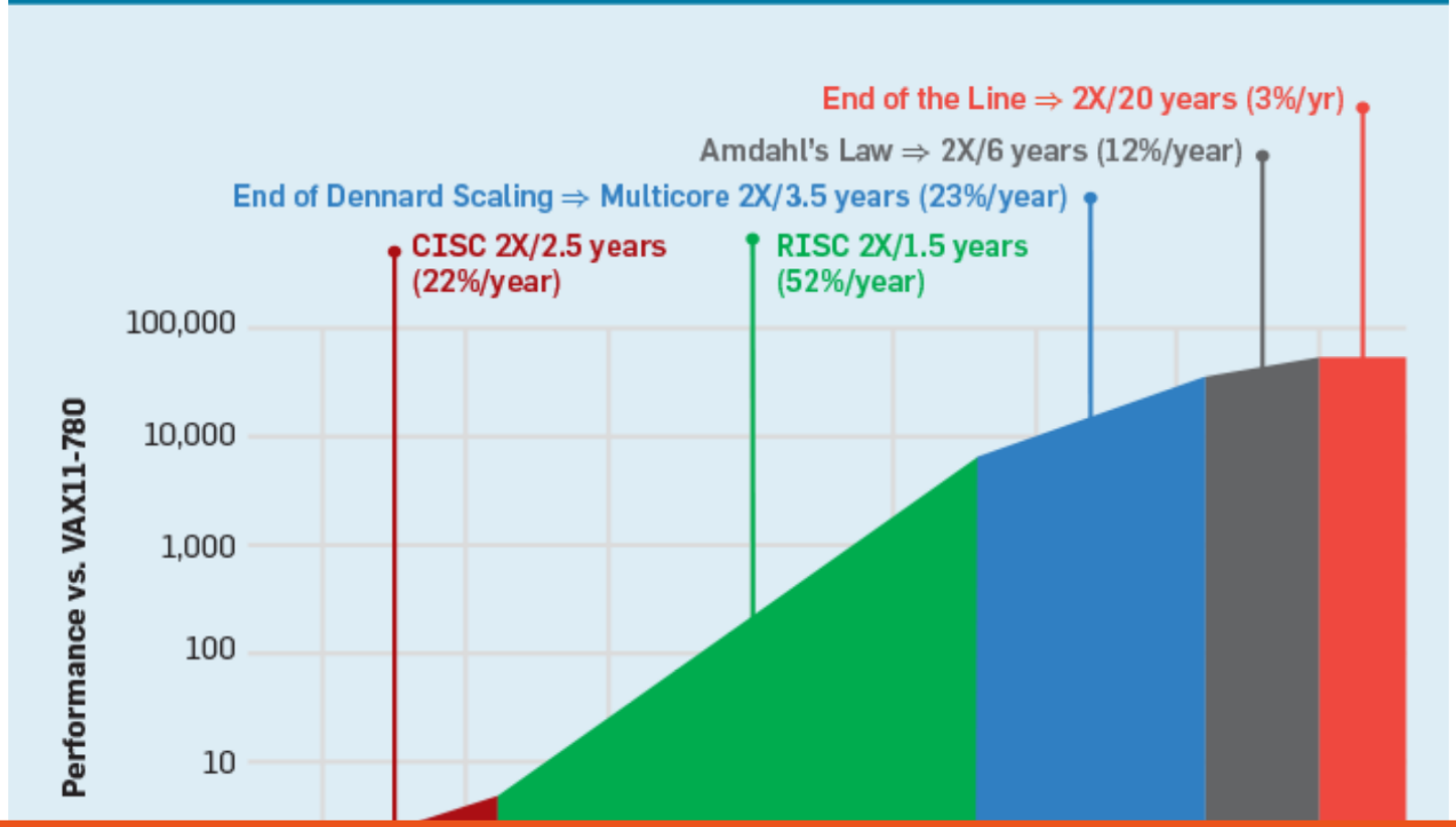


Compute requirements doubling every 3 months!

AI and Compute (https://blog.openai.com/ai-and-compute/)

2.2 机器学习/深度学习框架发展历程

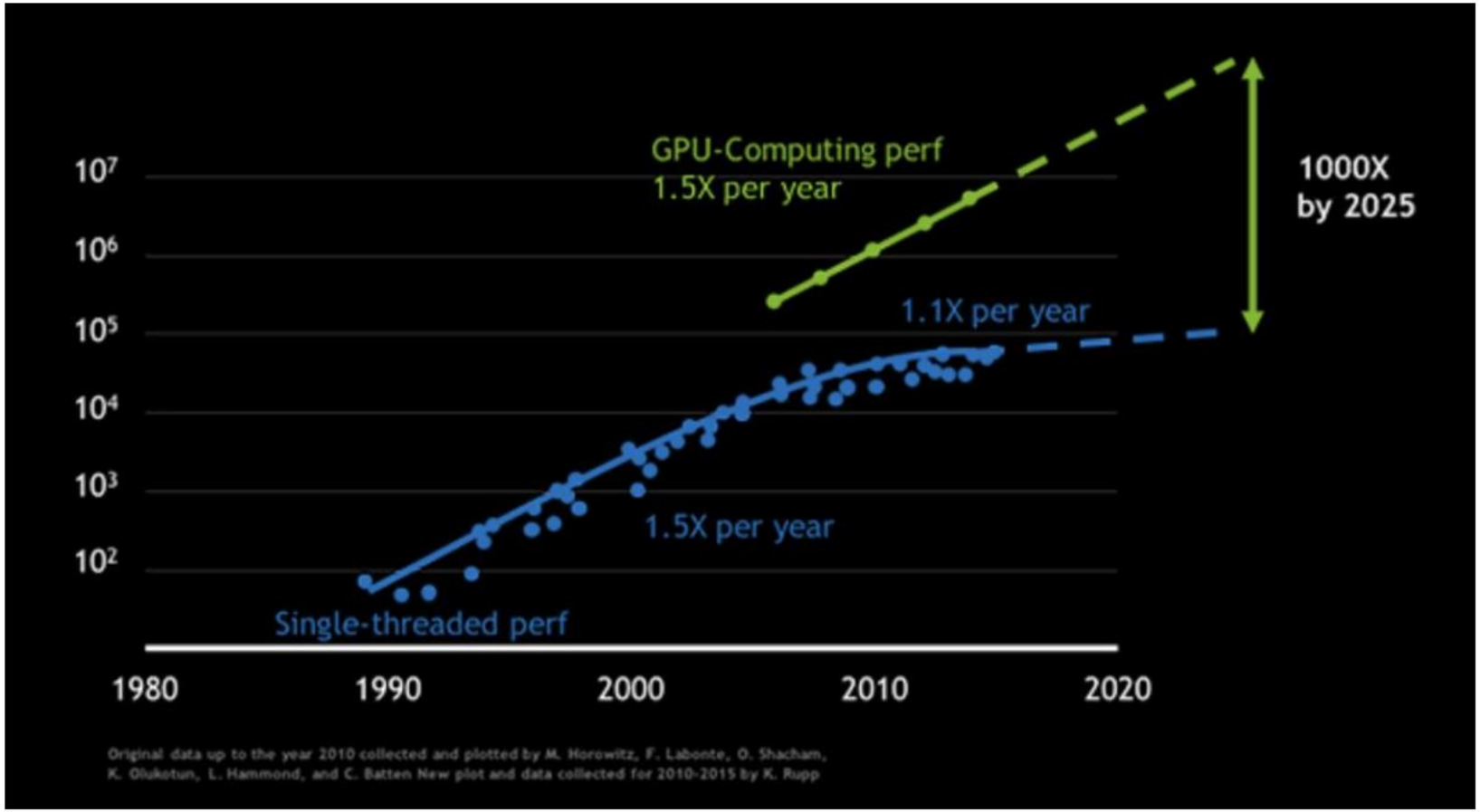
Figure 1. Following Hennessy and Patterson,[17] we plotted highest SPECCPUint performance per year for 32-bit and 64-bit processor cores over the past 40 years; the throughput-oriented SPECCPUint_rate reflects a similar profile, with plateauing delayed a few years.



Moore’s law is dead

2.2 机器学习/深度学习框架发展历程

GPU 性能提升仍然遵循摩尔定律



新一代的机器学习系统:

分布式

高效利用专用、异构硬件

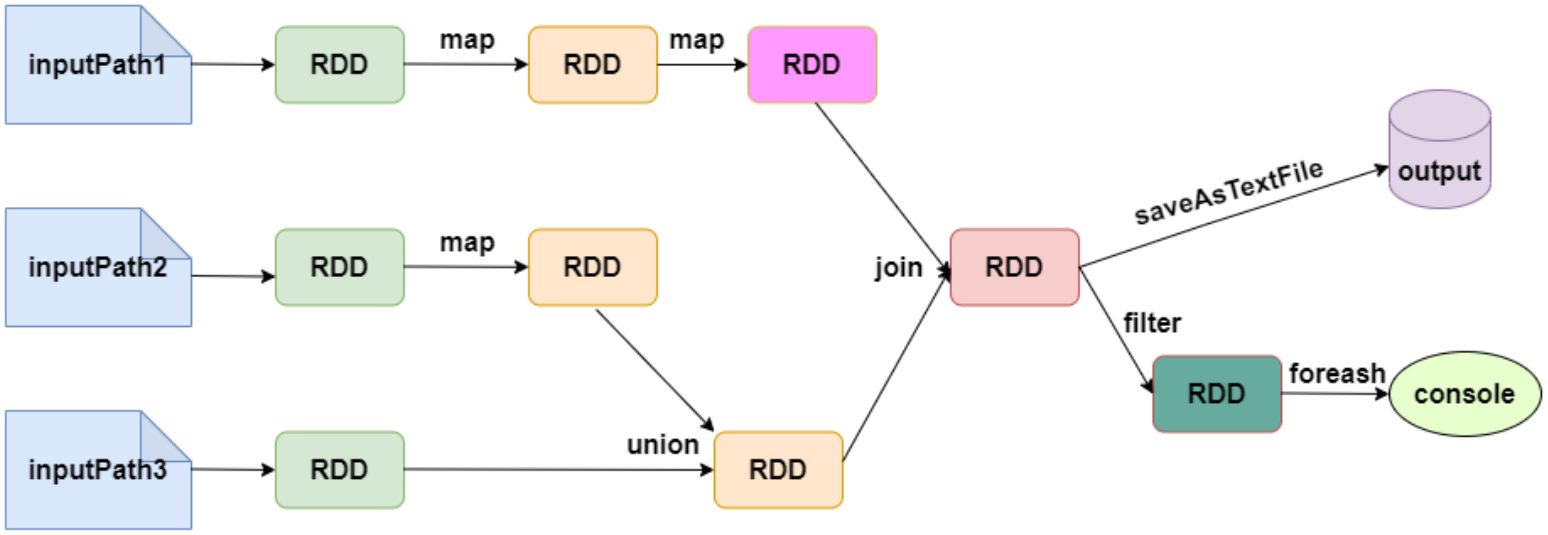
正在开发大量的神经网络加速器（例如 TPU）

2.3 分布式机器学习平台

- 分布式机器学习平台的两大基本计算模型
- 1. 基本数据流：Spark MLlib
- 2. 先进数据流：TensorFlow

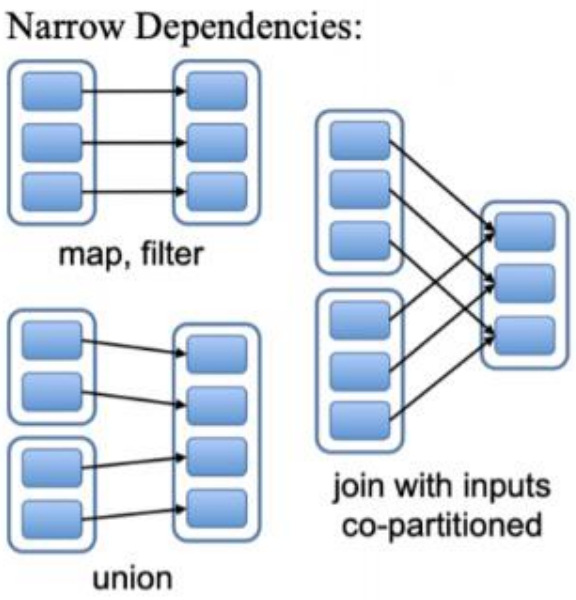
2.3 分布式机器学习平台

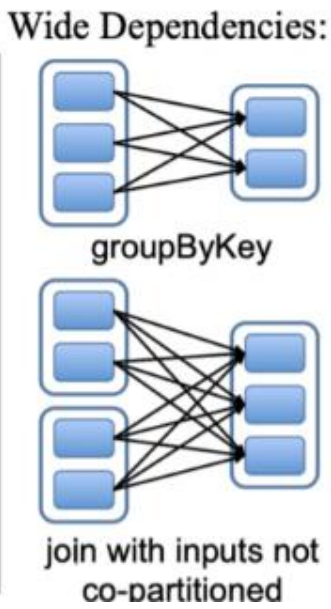
- 1. 基本数据流：Spark MLlib
- 基本数据流：使用有向无环图进行计算
- 计算被建模成有向无环图（DAG：directed acyclic graph），其中每一个顶点都代表一个弹性分布式数据集（RDD：Resilient Distributed Dataset），每一条边都代表对 RDD 的一个运算。

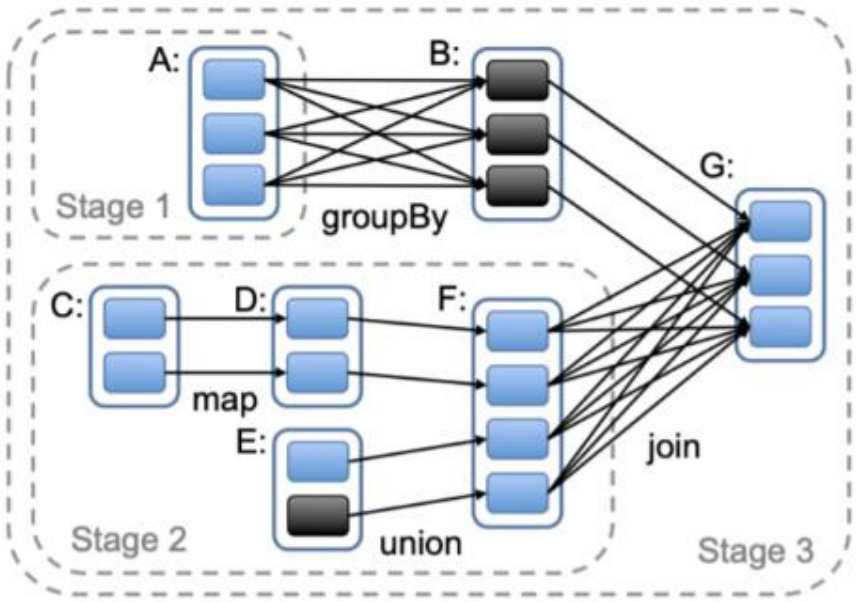


2.3 分布式机器学习平台

- 1. 基本数据流： Spark MLlib
- DAG 需要被编译为 stage。每个 stage 作为一系列并行运行的任务执行。简单狭窄的依赖关系有利于高效执行，而宽广的依赖关系会引入瓶颈。

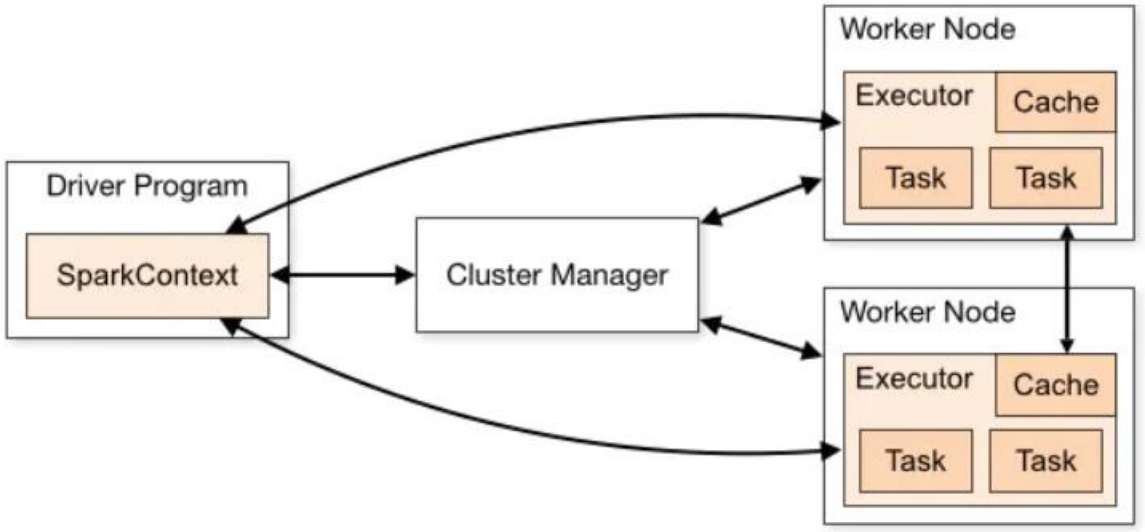






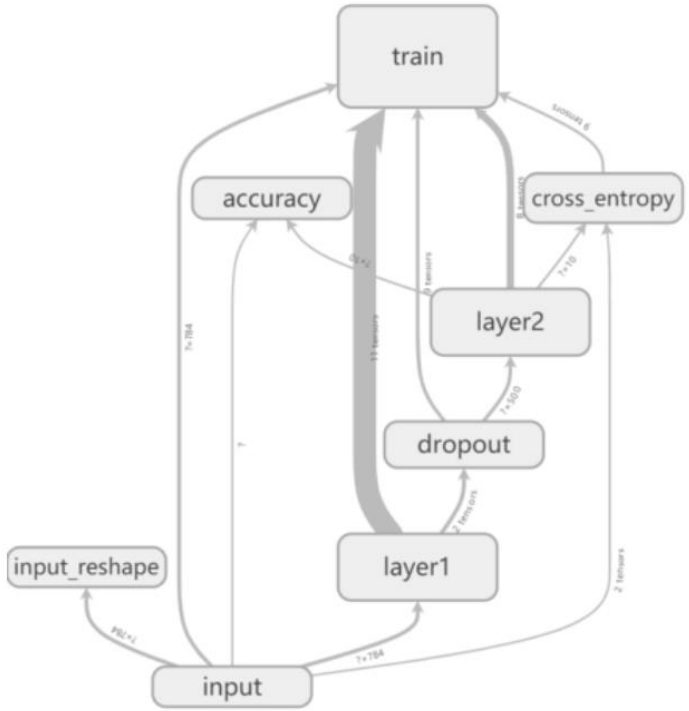
2.3 分布式机器学习平台

- 1. 基本数据流： Spark MLlib
- MLlib 中的分布式执行是采用master-worker架构，将 DAG stage 分割到不同的机器上执行的。驱动器（driver）负责调度任务，而工作器（worker）负责执行任务。
- 在MLlib for Spark中，Spark 将机器学习模型参数存储在驱动器节点，工作器与驱动器通信从而在每次迭代后更新这些参数。



2.3 分布式机器学习平台

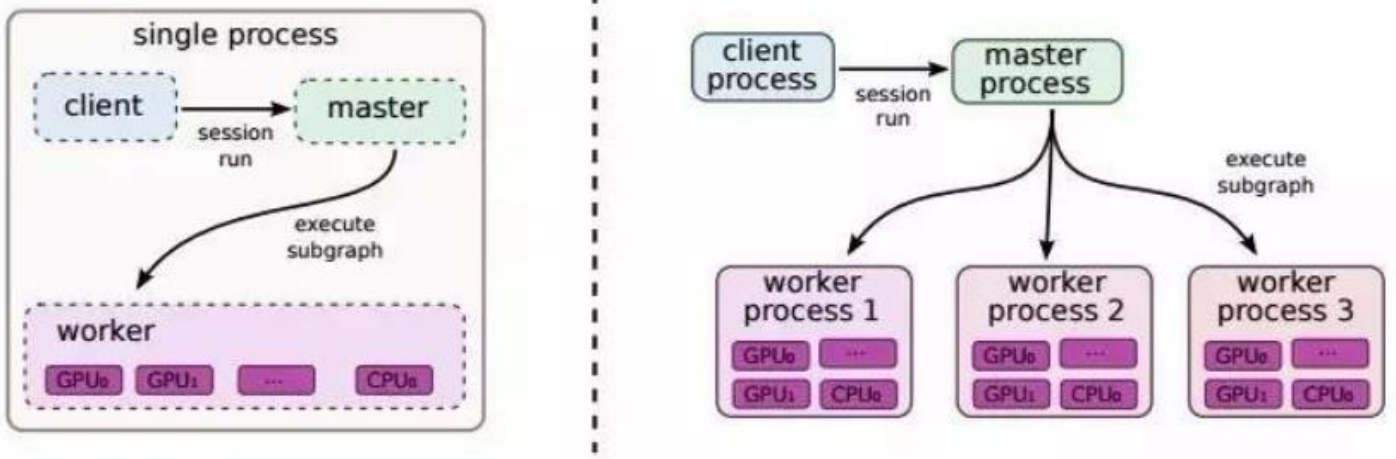
- 2. 先进数据流：TensorFlow
- 先进数据流：支持可变状态和循环的数据流图，无需是DAG
  - 节点：代表各种操作，例如数据填充、变量读写、数学运算、结果输出等。
  - 有向边：描述节点间的依赖关系，边上流动（flow）的数据代表一个batch训练过程中的临时变量（多维数据数组）。



2.3 分布式机器学习平台

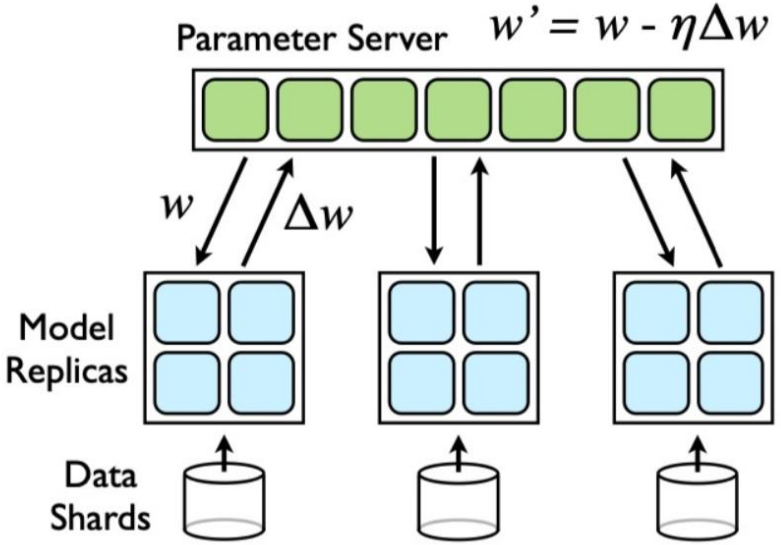
2. 先进数据流：TensorFlow

- worker:
  - TensorFlow由一个master进程和多个worker进程组成，master和worker之间通过接口通信，每个worker负责管理一个或者多个device（包含CPU和GPU）。
- cluster:
  - 为分布式执行的任务集，由一个或者多个job组成，每个job由多个task组成，每个task由一个worker执行。



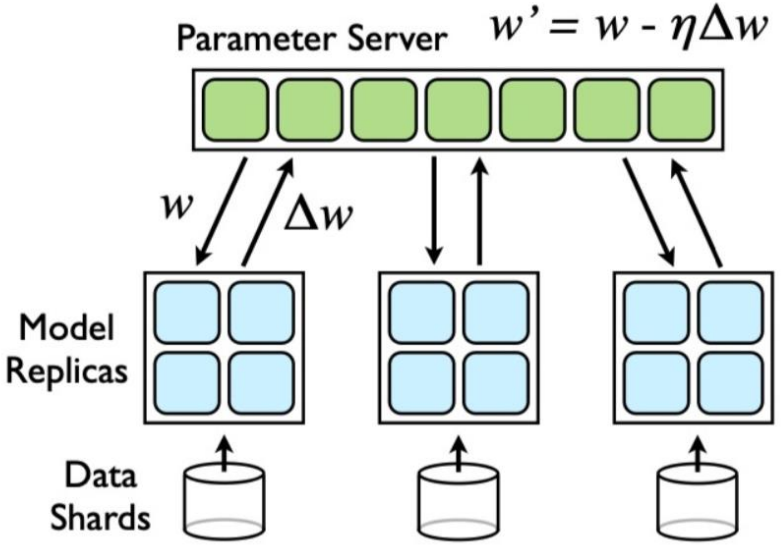
2.3 分布式机器学习平台

- 迭代计算模型： Petuum
- 参数服务器：
  - 主要用于实现不同节点之间的数据共享。
  - 参数服务器相当于是独立于所有节点的一个公共容器，可以将数据存储在该容器中，被不同的节点调用，不同的节点也可以往其中存储数据。



2.3 分布式机器学习平台

- 迭代计算模型： Petuum
- 借助参数服务器模型实现了机器学习的分布式异步计算
  - 运行时，训练数据被分为多个子集，在每个子集上运行模型的多个副本，模型通过集中式的参数服务器通信，参数服务器存放了模型的全部参数和状态。
  - 每个模型副本的计算完全独立，异步完成参数的拉取和更新。



2.4 Spark MLlib、TensorFlow框架比较

	Spark MLlib	TensorFlow
类型	机器学习	深度学习
支持语言	Scala、Java、Python	Python、C++
设计核心	基于RDD的有向无环图	基于数据流的计算图
建模方法	基本数据流	先进数据流
分布式计算实现方法	进程模型+调度系统	PS-worker
优点	高性能、易上手、可扩展性好、能够快速处理大规模数据	有深厚的灵活性、真正的可移植性、自动微分功能
缺点	支持的机器学习算法有限	调试困难、执行性能并无优势
应用	复杂的批数据处理，分类、回归、聚类、降维等机器学习应用场景	计算机视觉、自然语言处理和语音识别等领域

2.5 MLlib

- MLlib是Spark的机器学习库
  - 采用了一系列API定义并标准化了机器学习工作流，包含了数据收集、预处理、特征抽取、特征选择、模型拟合、模型验证、模型评估等一系列阶段。
- 特点
  - 采用较为先进的迭代式、内存存储的分析计算，使得数据的计算处理速度大大高于普通的数据处理引擎。
  - 提供了常用数据挖掘算法的分布式实现功能。

2.5 MLlib

组件

- ML 算法：包括了分类、回归、聚类、降维、协同过滤
- Featurization特征化：特征抽取、特征转换、特征降维、特征选择
- Pipelines管道：tools for constructing, evaluating, and tuning ML Pipelines
- Persistence持久化：模型的保存、读取、管道操作
- Utilities：提供了线性代数、统计学以及数据处理工具

2.5 MLlib

MLlib支持的多种数据模型

- 本地向量
  - 稠密向量（DenseVector）和稀疏向量（SparseVector）
  - 主要向Spark提供一组可进行操作的数据集合
- 标注点
  - 一种带有标签（Label/Response）的本地向量
  - 标注点类型在回归（Regression）和分类（Classification）问题上均可使用
- 本地矩阵
  - 稠密矩阵（DenseMatrix）和稀疏矩阵（Sparse Matrix）
  - 具有整型的行、列索引值和双精度浮点型的元素值
  - 将数据结合以矩阵形式存储在本地计算机中
- 分布式矩阵
  - 将矩阵集合以矩阵形式存储在分布式计算机中

$$\text{dense}: \underbrace{1.}\ \underbrace{0.}\ \underbrace{0.}\ \underbrace{0.}\ \underbrace{0.}\ \underbrace{0.}\ \underbrace{3.}$$

$$\text{sparse}: \begin{cases} \text{size}: 7 \\ \text{indices}: \underbrace{0}\ \underbrace{6} \\ \text{values}: \underbrace{1.}\ \underbrace{3.} \end{cases}$$

Training set:

- 12 million examples
- 500 features
- sparsity: 10%

	dense	sparse
storage	47GB	7GB
time	240s	58s

40GB savings in storage, 4x speedup in computation

```
Import org.apache.spark.mllib.linalg.Vectors
Import org.apache.spark.mllib.regression.LabeledPoint

// Create a labeled point with a positive label and a dense feature vector.
val pos = LabeledPoint(1.0, Vectors.dense(1.0, 0.0, 3.0))

// Create a labeled point with a negative label and a sparse feature vector.
val neg = LabeledPoint(0.0, Vectors.sparse(3, Array(0, 2), Array(1.0, 3.0)))
```

2.5 MLlib

- MLlib支持的主要算法

名称	说明
数学统计计算库	基本统计量、相关分析、随机数产生器、假设检验等
算法评测	AUC、准确率、召回率、F-Measure 等
机器学习算法	分类算法、回归算法、聚类算法、协同过滤等

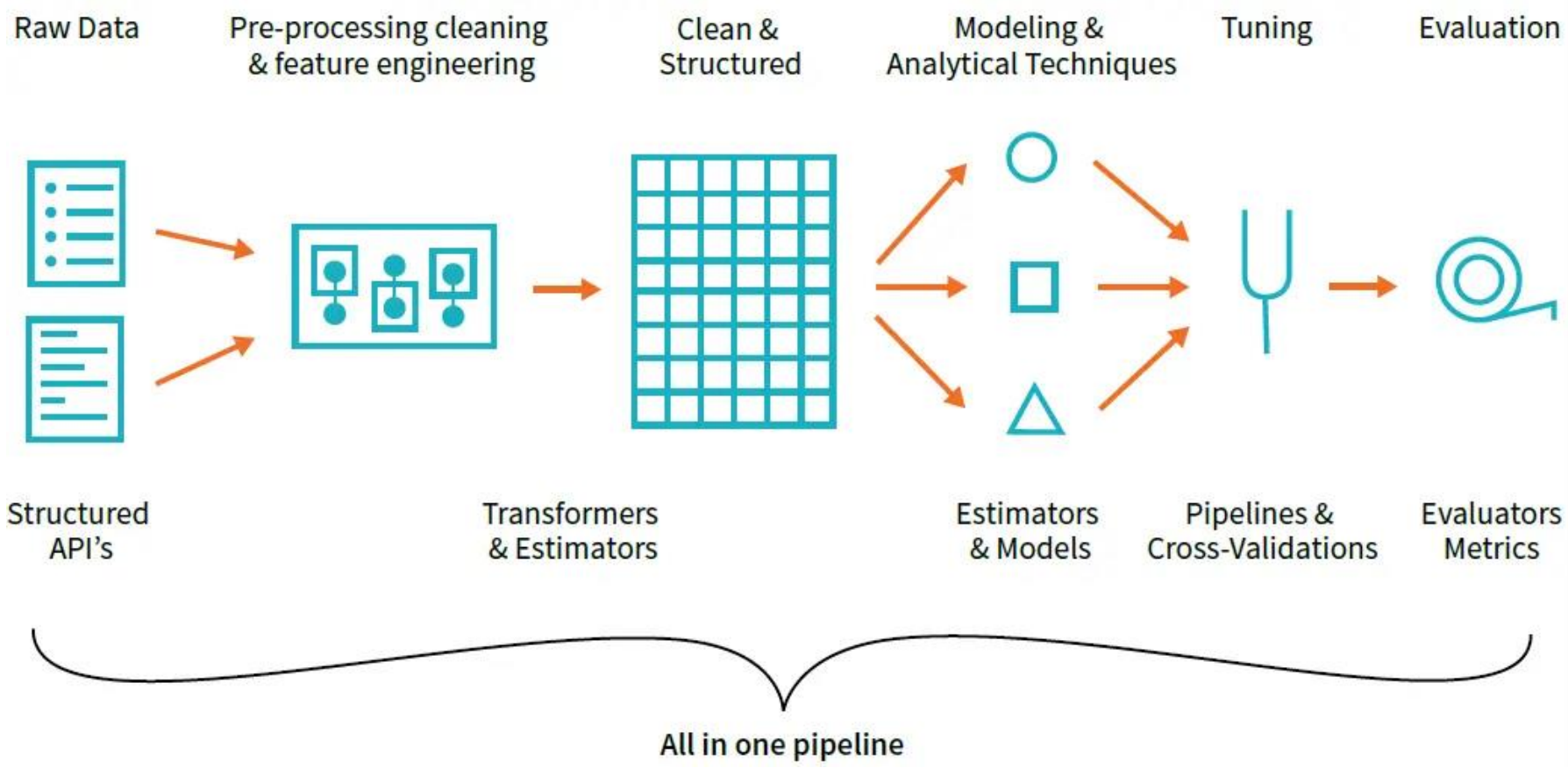
- 其中，机器学习算法主要有：

	离散数据	连续数据
监督学习	Classification、LogisticRegression(with Elastic-Net)、SVM、DecisionTree、RandomForest、GBT、NaiveBayes、MultilayerPerceptron、OneVsRest	Regression、LinearRegression(with ElasticNet)、DecisionTree、RandomFores、GBT、AFTSurvivalRegression、IsotonicRegression
无监督学习	Clustering、KMeans、GaussianMixture、LDA、PowerIterationClustering、BisectingKMeans	Dimensionality Reduction, matrix factorization、PCA、SVD、ALS、WLS

2.5 MLlib 主要概念

Pipelines

- 对分布式机器学习过程进行模块化的抽象，使得多个算法合并成一个Pipeline或者工作流



2.5 MLlib 主要概念

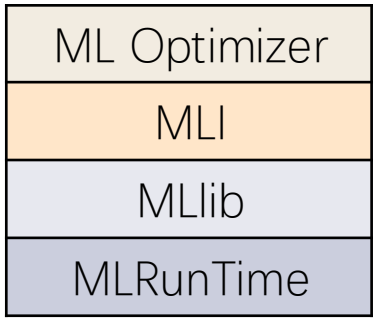
Pipelines

- DataFrame：数据存储结构
  - 为Spark SQL中的DataFrame结构,其列可以存储特征向量，标签，以及原始的文本，图像。
- Transformer：一种算法模型
  - 直接对DF进行变换
- Estimator：一种算法模型
  - 需要训练后才能对DF进行变换
- Pipeline：组织多个ML模型
  - 多个Transformer和Estimator形成算法工作流
- Parameter：存储参数
  - 用户通过这些参数来配置和调节模型。即在一个Pipeline对象内的所有Transformer和Estimator都共享一个Parameter对象

2.5 MLlib 简介

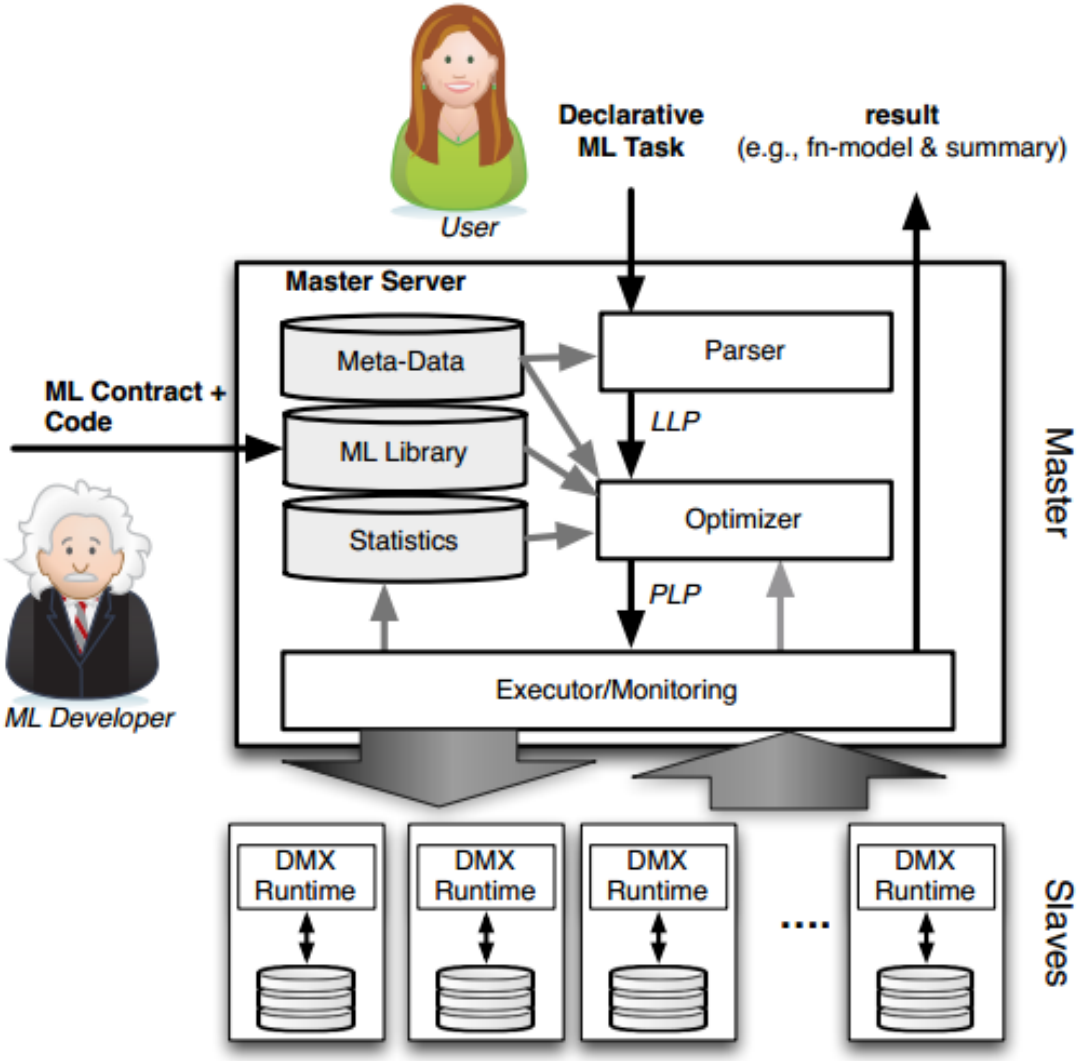
MLBase架构

- MLlib是Spark机器学习库，它是MLBase的一部分，MLBase一共分为以下4部分
- MLRunTime
  - 是基于Spark计算框架，将Spark的分布式计算应用到机器学习领域。
- MLlib
  - Spark实现一些常见的机器学习算法和实用程序。
- MLI
  - 是一个进行特征提取的和高级ML编程抽象的算法实现的API平台。
- ML Optimizer
  - 会选择它认为最适合的已经在内部实现好了的机器学习算法和相关参数来处理用户输入的数据，并返回模型或者其他的帮助分析结果。



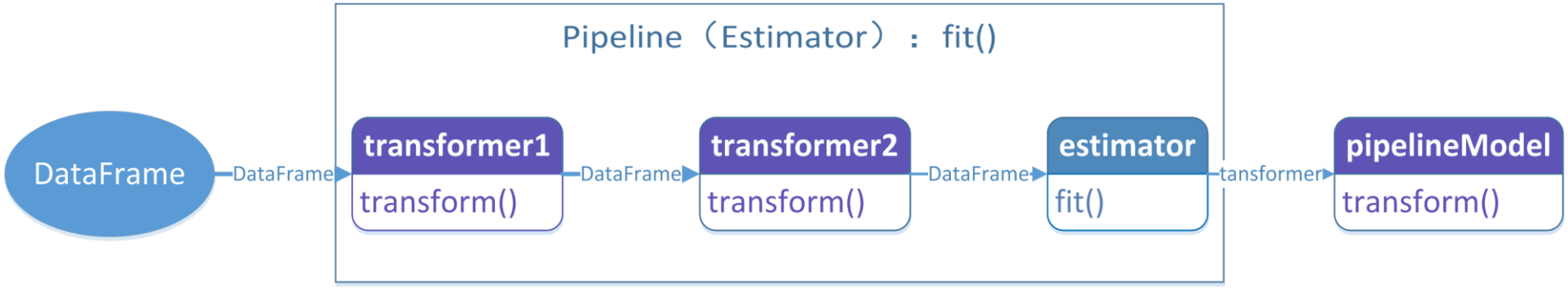
2.5 MLlib 简介

- MLBase架构

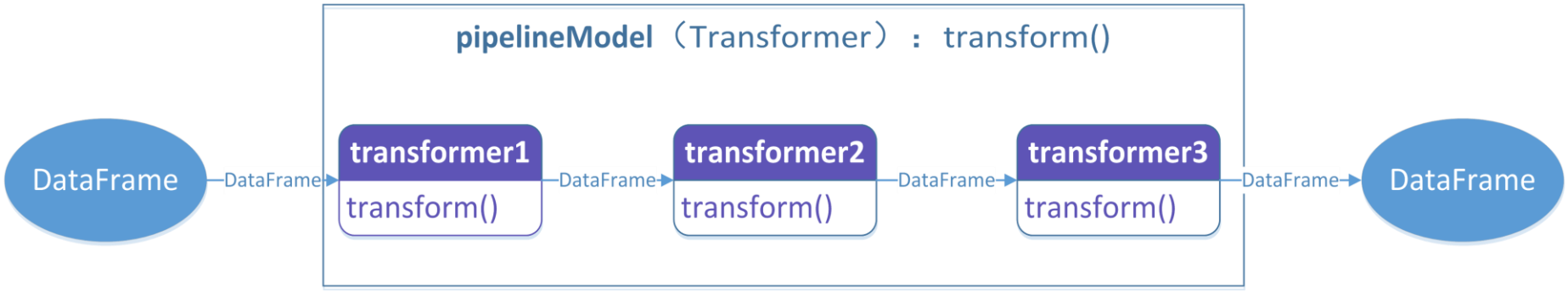


2.5 MLlib 工作流程

- 1.数据准备阶段
- 2.训练模型评估阶段



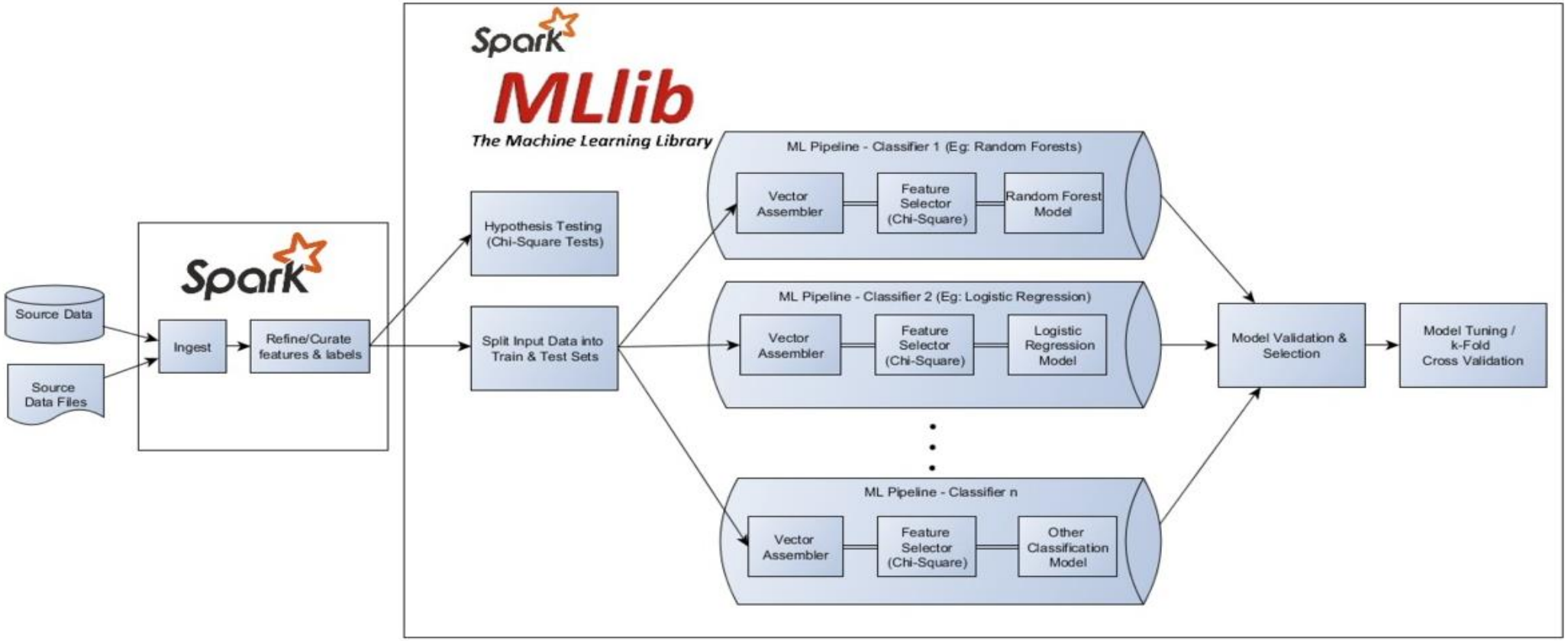
- 3.部署预测阶段



2.5 MLlib 工作流程

多模型集成

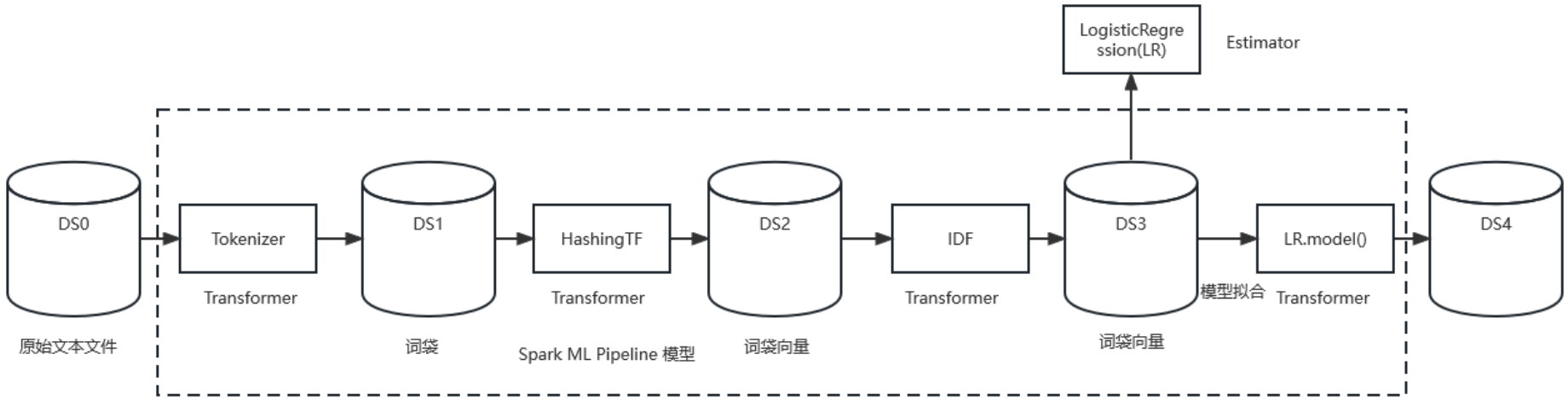
- 摄取数据，执行特征工程，训练三种机器学习模型，选择最佳模型以及调整最佳模型的端到端过程



2.5 MLlib 应用实例

文本情感预测

- 应用场景
  - 移动互联网的发展和普及已达到空前规模，亿万用户在互联网上可以获得信息、交流信息，发表自己的观点和分享自己的体验，表达各种情感和情绪，如批评、赞扬以及喜、怒、哀、乐等。
  - 推荐理由、推送内容、广告文案、精选评价标签、商家运营、对话系统等场景都需要识别文本在整体或属性上的情感倾向，从而分析用户的情感倾向。
- 实现过程
  - 准备数据
  - 训练模型
  - 评估模型
  - 预测



2.5 MLlib 应用实例

1.准备数据

- IMDB电影评论数据集（txt文件）
  - 包含50,000个评论，其中25,000个评论用于训练模型，另外25,000个评论用于测试模型。每个评论都有一个标签，标签为正面或负面，表示评论是正面的还是负面的。
  - 每个评论都以数字1或数字0开头，数字1表示该评论为正面评论，数字0表示该评论为负面评论。数字后面紧跟着一个制表符（\t）和评论文本。例如：
    - 1 This movie was absolutely amazing!
    - 0 I couldn't stand this film. It was terrible.

2.5 MLlib 应用实例

1.准备数据

```
def load_data(file_path):
    raw_text = spark.read.text(file_path).toDF("text")
    html_regex = re.compile("<.*?>")
    df = raw_text.select("text", (raw_text["text"].rlike(html_regex)).alias("hasHTML"))
    df = df.filter("hasHTML=false")
    # 分离正面和负面评论
    pos_df = df.filter("text rlike '1\t'")
    neg_df = df.filter("text rlike '0\t'")
    pos_df = pos_df.select("text", np.ones(pos_df.count()).alias("label"))
    neg_df = neg_df.select("text", np.zeros(neg_df.count()).alias("label"))
    return pos_df.union(neg_df)
```

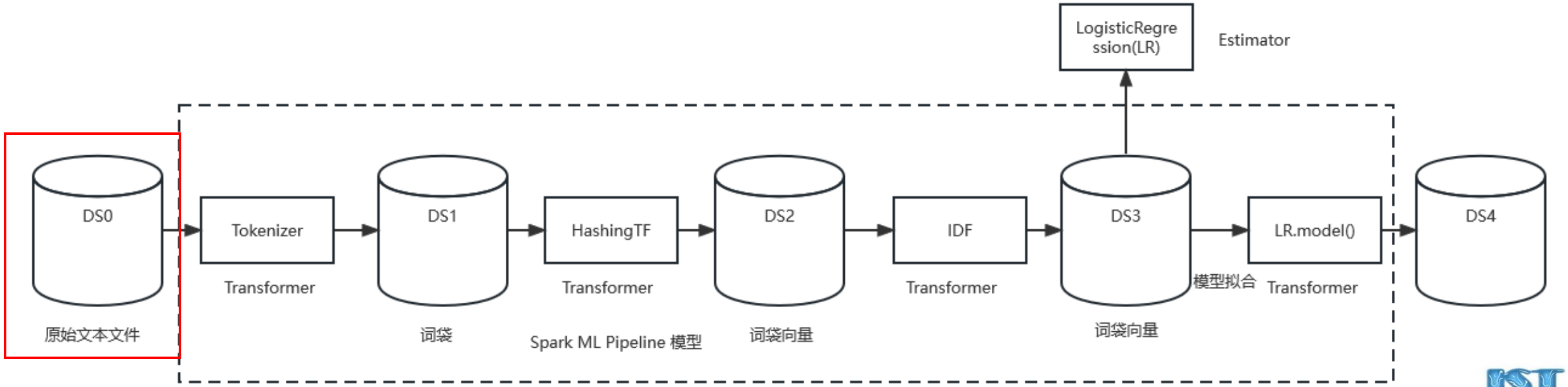
- df.filter("text rlike '1'")： 从DataFrame中筛选出包含1的评论文本，并返回一个新的DataFrame。
- pos_df.select("text")选取了数据集中的"text"列，即电影评论的文本内容。np. ones(pos_df.count()).alias("label")创建了一个名为"label"的新列，其中每个值都被初始化为1。最后，将"text"列和"label"列拼接起来，构成了一个包含文本和标签的新DataFrame。

2.5 MLlib 应用实例

1.准备数据

```
# 加载数据集并进行预处理
data = load_data("path_to_imdb_data.txt")

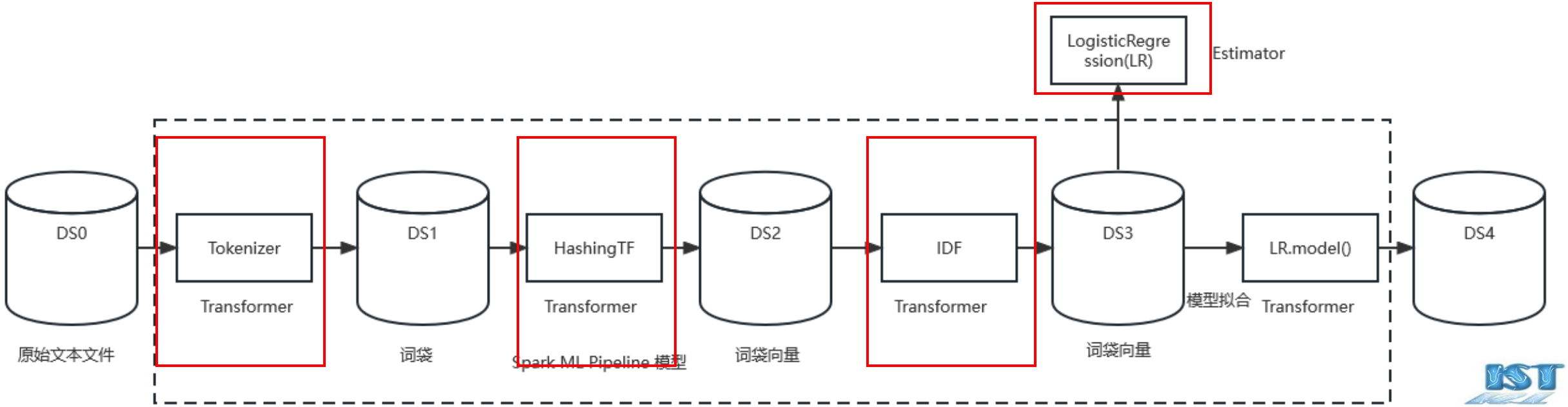
# 将数据集拆分为训练集和测试集
(train_set, test_set) = data.randomSplit([0.8, 0.2], seed=1234)
```



2.5 MLlib 应用实例

2.训练模型

```
# 使用Spark的ML库进行特征提取和模型训练
tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(numFeatures=1000, inputCol=tokenizer.getOutputCol(),
outputCol="rawFeatures")
idf = IDF(inputCol=hashingTF.getOutputCol(), outputCol="features", minDocFreq=5)
lr = LogisticRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)
lr_model = lr.fit(training_data)
```



2.5 MLlib 应用实例

2.训练模型

```
# 使用Spark的ML库进行特征提取和模型训练
tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(numFeatures=1000, inputCol=tokenizer.getOutputCol(),
outputCol="rawFeatures")
```

- TF(Term Frequency)：表示某个单词或短语在某个文本中出现的频率

$$TF_{i,j} = \frac{n_{i,j}}{\sum n_{i,j}}$$

分子表示该单词在文本中出现的次数，分母表示在文本中所有单词的出现次数之和

2.5 MLlib 应用实例

➢ 2.训练模型

```
# 使用Spark的ML库进行特征提取和模型训练
tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(numFeatures=1000, inputCol=tokenizer.getOutputCol(),
outputCol="rawFeatures")
```

- Tokenizer函数：将文本拆分成单词
  - Input：输入文本
  - Output：输出单词
- HashingTF函数：使用了特征哈希的方法将词映射到固定长度的特征向量
  - numFeatures：特征的数量
  - inputCol：从文本中拆分出的单词
  - outputCol：TF计算出的单词频率

2.5 MLlib 应用实例

➢ 2.训练模型

```
# 使用Spark的ML库进行特征提取和模型训练
idf = IDF(inputCol=hashingTF.getOutputCol(), outputCol="features", minDocFreq=5)
```

- IDF(Inverse Document Frequency，逆向文件频率)：在所有文本中，若包含某个单词的文本越少，即单词个数越少，则IDF值就越大，则说明某个单词具有良好的类别区分能力。某一个单词的IDF可有总文本数除以包含该单词文本的数目，再将得到的商取以10为底的对数

$$IDF_i = lg\frac{|D|}{|\{j : t_i \subset d_j\}|}$$

其中，|D|表示语料库中的文本总数， 分母表示包含单词的文本数目

- IDF 函数：接收HashingTF产生的特征数据，并生成IDF模型，后续被调用
  - inputCol：TF计算出的单词频率
  - outputCol：IDF计算出的逆向文本频率
  - minDocFreq：单词的最低逆向文本频率

2.5 MLlib 应用实例

2.训练模型

```
# 使用Spark的ML库进行特征提取和模型训练
lr = LogisticRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)
lr_model = lr.fit(training_data)
```

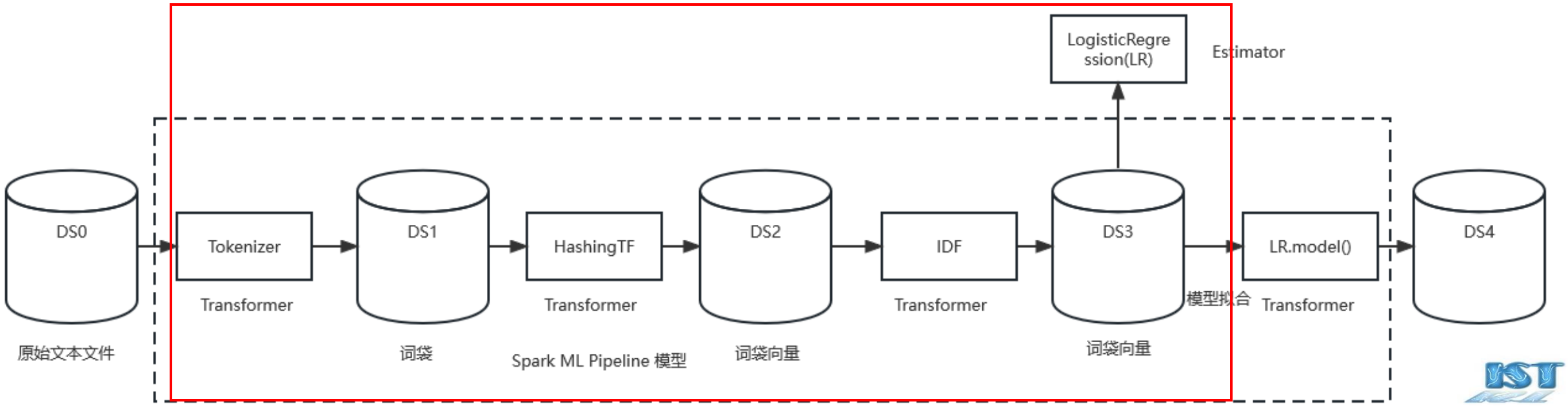
- LR (逻辑回归)：可以根据输入特征预测一个二元或多元分类结果的概率，并将其转换为类别。
- LogisticRegression函数：
  - maxIter：算法的最大迭代次数
  - regParam：正则化参数，用于控制过拟合
  - elasticNetParam：用于控制L1和L2正则化之间的权衡

2.5 MLlib 应用实例

2.训练模型

```
# 将特征提取和模型训练构成一个pipeline
from pyspark.ml import Pipeline
pipeline = Pipeline(stages=[tokenizer, hashingTF, idf,lr])

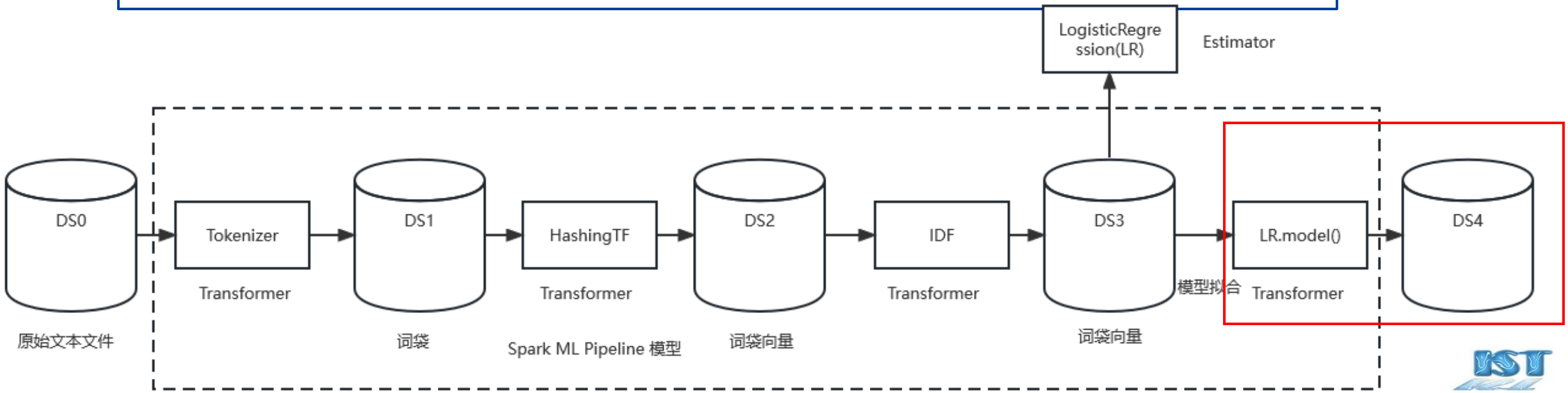
# 训练模型
model = pipeline.fit(train_set)
```



2.5 MLlib 应用实例

3.评估模型

```
# 评估模型在测试集上的性能
predictions = model.transform(test_set)
evaluator = MulticlassClassificationEvaluator(predictionCol="prediction")
accuracy = evaluator.evaluate(predictions)
print("Accuracy on test set: ", accuracy)
# 保存模型
model_path = "path_to_save_model"
model.save(model_path)
```



2.5 MLlib 应用实例

➢ 4.预测

```
new_data = spark.createDataFrame([
 (0, "This movie was great! The acting was superb and the storyline was very engaging."),
(1, "This movie was terrible! The acting was awful and the plot was very boring.") ],
["id", "text"])

predictions = model.transform(new_data)
predictions.select("id", "text", "prediction").show()
```



```
+---+--------------------+----------+
| id|                text|prediction|
+---+--------------------+----------+
|  0|This movie was gr...|       1.0|
|  1|This movie was te...|       0.0|
+---+--------------------+----------+
```

目录

- 1 数据智能计算架构回顾
- 2 分布式机器学习框架
- 5 小结

小结

- 数据智能应用的机器学习框架
  - 机器学习vs深度学习
  - 基本数据流vs先进数据流
  - Mllib架构